

Informe de rendimiento

Sistema ALFA-Sentinel — Detección temprana de ransomware basada en archivos señuelo y análisis heurístico de procesos

Preparado para: SIATEC | Fecha de las mediciones: 22 de septiembre de 2026 | Fecha del informe: 22 de septiembre de 2026

1. Resumen ejecutivo

Se midió el rendimiento de las 14 llamadas a la API que la consola web de ALFA-Sentinel realiza al navegar entre módulos (Panel de control, Endpoints, Alertas, Incidentes, Honeyfiles, Reglas heurísticas, Respuesta, Reportes, Administración y Perfil), comparando el acceso directo al backend FastAPI y el acceso a través del proxy de desarrollo de Vite que usa el frontend React.

Resultado principal: las 14 llamadas medidas caen dentro de la categoría «Rápido» ($p95 \leq 200$ ms) tanto en acceso directo como a través de Vite. La mediana promedio del backend es de 52.8 ms y el p95 promedio de 133.7 ms; el punto más lento observado es *Incidentes* con un p95 de 178.8 ms, aun así por debajo del umbral de 200 ms.

Lo que este informe NO incluye todavía: la huella de CPU y memoria del agente instalado en el endpoint («¿es un agente ligero?»). La herramienta para medirla ya se construyó (`tests/performance/benchmark_agent_footprint.py`) pero requiere ejecutarse en un equipo real con el agente corriendo; el entorno desde el que se preparó este informe no tiene permisos ni conectividad para levantar el backend, PostgreSQL y el agente por sí mismo. La sección 6 detalla esto y el análisis cualitativo disponible mientras tanto.

2. Alcance y metodología

2.1 Arquitectura evaluada

ALFA-Sentinel se compone de tres piezas: un **agente** Python instalado en cada endpoint (vigila archivos señuelo, carpetas de usuario y procesos, con adaptadores separados para Windows —ETW— y Linux —fanotify—), un **backend** FastAPI con PostgreSQL que centraliza reglas, alertas, incidentes y políticas, y una **consola web** en React/Vite que consume la API del backend.

2.2 Qué se midió

- **Latencia de API (backend directo):** 20 muestras por endpoint tras 3 peticiones de calentamiento, contra `http://127.0.0.1:8000`.
- **Latencia de API (a través de Vite):** mismas condiciones, contra `http://localhost:5173`, para aislar cuánto añade el proxy de desarrollo.
- **Login:** 261.9 ms (directo) vs. 380.5 ms (vía Vite); incluye la actualización real de `last_login_at`, igual que un inicio de sesión normal.

Herramienta usada: `tests/performance/benchmark_navigation.py` (ya existente en el repositorio). Métricas reportadas por endpoint: mediana, percentil 95, promedio, mínimo y máximo.

2.3 Limitaciones de esta ronda de medición

- Una sola sesión, un solo equipo, con el backend corriendo en modo desarrollo (`uvicorn --reload`), no en una configuración de producción.
- No se probó con un volumen de datos realista en PostgreSQL (cantidad de endpoints, alertas e incidentes de un despliegue real).
- No se midió concurrencia: las mediciones son de un único usuario/sesión, no de varios agentes o analistas actuando al mismo tiempo.
- No se midió la huella de CPU/RAM del agente (ver sección 6).

3. Resultados — latencia de API

3.1 Backend directo (puerto 8000)

Clasificación según la propia escala diagnóstica del proyecto (no es un requisito formal): hasta 200 ms = Rápido, 200–500 ms = Aceptable, 500–1000 ms = Lento, más de 1000 ms = Muy lento; basada en el p95 de cada endpoint.

Pantalla / endpoint	Mediana (ms)	p95 (ms)	Prom. (ms)	Mín (ms)	Máx (ms)	Clasif.
Dashboard / overview	64.3	79.4	65.9	56.5	79.4	Rápido
Dashboard / actividad 24 h	51.5	66.8	53.1	45.1	66.8	Rápido
Endpoints	62.4	162.4	71.1	49.3	162.4	Rápido
Alertas	59.8	152.4	64.7	47.8	152.4	Rápido
Incidentes	69.2	178.8	74.7	58.4	178.8	Rápido
Honeyfiles	52.9	160.8	58.3	44.4	160.8	Rápido
Reglas heurísticas	46.8	77.2	51.6	42.6	77.2	Rápido
Acciones de respuesta	58.6	158.4	71.0	47.1	158.4	Rápido
Reportes	52.7	159.2	65.1	40.1	159.2	Rápido
Administración / usuarios	44.4	165.2	54.3	38.2	165.2	Rápido
Administración / configuración	38.7	143.5	46.5	34.0	143.5	Rápido
Administración / auditoría	47.4	71.9	47.5	37.0	71.9	Rápido
Perfil	39.4	143.0	50.6	34.6	143.0	Rápido
Alertas abiertas (poll 3 s)	51.3	153.3	54.9	42.4	153.3	Rápido

14 de 14 endpoints clasifican como «Rápido». Ninguno superó el umbral de 200 ms en p95. El endpoint más lento es *Incidentes* (178.8 ms de p95); el de mediana más baja es *Administración / configuración* (38.7 ms).

3.2 A través del proxy de Vite (puerto 5173)

Pantalla / endpoint	Mediana (ms)	p95 (ms)	Δ mediana vs. directo	Δ p95 vs. directo
Dashboard / overview	72.3	180.9	+8.1	+101.6
Dashboard / actividad 24 h	58.7	168.8	+7.2	+102.0
Endpoints	66.4	173.7	+4.0	+11.3
Alertas	60.7	71.8	+0.8	-80.7
Incidentes	69.9	177.1	+0.7	-1.7
Honeyfiles	58.5	72.6	+5.6	-88.3
Reglas heurísticas	63.1	160.3	+16.3	+83.1
Acciones de respuesta	61.2	87.5	+2.6	-70.9
Reportes	55.2	155.5	+2.5	-3.6
Administración / usuarios	51.4	170.4	+7.0	+5.2
Administración / configuración	46.1	163.8	+7.5	+20.3
Administración / auditoría	48.5	148.4	+1.1	+76.5
Perfil	52.0	173.6	+12.6	+30.6

Pantalla / endpoint	Mediana (ms)	p95 (ms)	Δ mediana vs. directo	Δ p95 vs. directo
Alertas abiertas (poll 3 s)	58.4	156.5	+7.1	+3.3

El proxy de Vite añade, en promedio, +5.9 ms a la mediana y +13.5 ms al p95 respecto al backend directo. Es un costo esperable de un servidor de desarrollo y **no aplica a producción**: el frontend ya cuenta con un build estático (frontend/dist/) que se serviría sin este intermediario.

3.3 Comparación visual

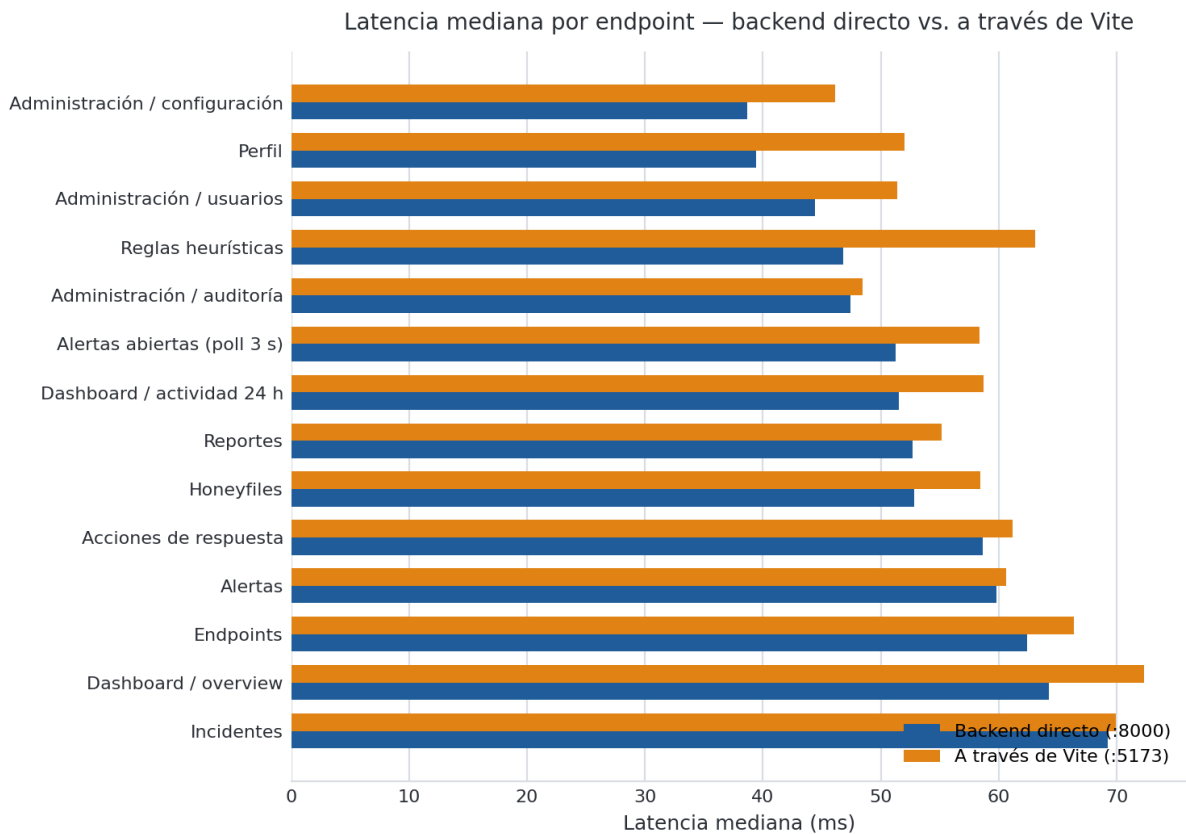


Figura 1. Latencia mediana por endpoint, backend directo vs. a través de Vite.

4. Huella de recursos del agente (CPU / RAM) — pendiente de medición real

Esta sección es la más relevante para decidir si el agente es «ligero» y no va a causar problemas en los equipos donde se instale — pero todavía no contiene datos medidos, solo el plan y un análisis de diseño. Se deja así, en vez de estimarla, para no presentar una cifra no verificada en un informe de caso de estudio.

4.1 Por qué no se midió en esta ronda

El agente necesita un backend alcanzable para autenticarse y aplicar políticas antes de iniciar sus hilos de monitorización, y ese backend necesita PostgreSQL. El entorno desde el que se preparó este informe no tiene permisos de administrador ni acceso de red para instalar PostgreSQL, FastAPI, uvicorn, psycpg2 o watchdog, y los entornos virtuales ya existentes en el repositorio (`agent/.venv`, `server/.venv`) son específicos de Windows y no corren en ese entorno. Medir esto con datos reales requiere ejecutarlo en el equipo Windows de destino.

4.2 Herramienta ya lista para completarla

Se construyó `tests/performance/benchmark_agent_footprint.py` (con su `README_agente.md`), que se engancha al proceso del agente por PID y lo muestrea con `psutil` — CPU, memoria residente y virtual, hilos, archivos abiertos y E/S de disco — en fases de «reposo» y «carga» (esta última coordinada con el simulador seguro de comportamientos tipo ransomware, `tools/simulator/alfa_ransomware_simulator.py`), y clasifica el resultado como ligero / aceptable / pesado.

4.3 Qué se puede decir hoy, solo a partir del código (no medido)

Estas observaciones son de diseño, no un resultado empírico, y deben confirmarse con el benchmark antes de tomarlas como base para desplegar en producción:

- Dependencias del agente reducidas a tres librerías (`psutil`, `watchdog`, `httpx`), sin frameworks pesados — apunta a una huella de instalación pequeña, aunque no dice nada sobre el consumo en ejecución.
- La vigilancia de archivos está basada en eventos (`watchdog` / `fanotify` en Linux, ETW en Windows) en vez de sondeo (polling) constante del sistema de archivos, lo que en general reduce el uso de CPU en reposo frente a un escáner que recorre el disco periódicamente.
- Existen varios hilos con sondeo periódico propio (heartbeat, sincronización de honeyfiles, sincronización de aislamiento, monitor de CPU) — su intervalo configurado es lo que más puede empujar el consumo en reposo hacia arriba, y es el primer lugar a revisar si el benchmark muestra un uso alto sin actividad.

5. Escenarios de implementación

Recomendaciones basadas en lo medido (latencia de API) y en la arquitectura del sistema. Los escenarios marcados **[pendiente]** dependen de completar la sección 4 antes de una decisión final.

Escenario	Viabilidad con lo medido hasta ahora	Condición para confirmar
PYME / oficina con red local propia y servidor centralizado	Buena — latencias de API muy por debajo de 200 ms en LAN, carga de un solo analista/turno típica de este tamaño.	Validar con el volumen real de endpoints y alertas de la organización.
Equipos de escritorio Windows y Linux de usuario final	Soportada por diseño (adaptadores ETW / fanotify), pero [pendiente] sin confirmar el costo real en CPU/RAM del agente sobre hardware de oficina típico.	Correr <code>benchmark_agent_footprint.py</code> en un equipo representativo antes de desplegar a gran escala.
Equipos ya cargados (estaciones de diseño/CAD, servidores de aplicaciones) o con recursos limitados (equipos antiguos, netbooks)	[pendiente] No recomendable decidir todavía — es justamente donde un agente pesado causaría más problemas.	Exigir el resultado de la sección 4 (clasificación «ligero») antes de instalar ahí.
Redes con muchos endpoints reportando simultáneamente (decenas o cientos de agentes)	[pendiente] No evaluado — todas las mediciones son de una sola sesión/usuario contra el backend.	Prueba de carga concurrente contra el backend (múltiples agentes/heartbeats simultáneos), no realizada en este informe.
Laptops o equipos con conectividad intermitente al servidor central	Riesgo arquitectónico, no de rendimiento: el agente depende de heartbeat y sincronización periódica con el backend para políticas y reporte de aislamiento.	Definir comportamiento esperado del agente sin conexión prolongada antes de desplegar en equipos móviles.
Entorno de producción del backend (fuera de --reload de desarrollo)	Se espera igual o mejor rendimiento que lo medido (--reload añade overhead de desarrollo), pero no se midió así.	Repetir <code>benchmark_navigation.py</code> contra un despliegue de producción real.

6. Conclusiones y próximos pasos

La capa de API del sistema (backend FastAPI + PostgreSQL, consumida por la consola web) responde rápido bajo las condiciones probadas: 14 de 14 pantallas clasifican como «Rápido», con una mediana promedio de 52.8 ms. Esto respalda la viabilidad del sistema para un despliegue de tamaño PYME sobre red local, siempre que se confirme con datos reales de volumen y concurrencia.

La pregunta que motivó este informe — «¿el agente es ligero, no va a causar problemas en los equipos donde se instale?» — sigue abierta. Es la pieza que falta para decidir con seguridad en qué escenarios de la sección 5 se puede desplegar sin reservas.

Próximos pasos recomendados, en orden

- Ejecutar `tests/performance/benchmark_agent_footprint.py` en un equipo Windows real, con el agente enrolado y corriendo contra el backend, incluyendo una fase de carga con el simulador de ransomware.
- Repetir `benchmark_navigation.py` con un volumen de datos representativo (endpoints, alertas e incidentes reales, no los de una base recién sembrada) y, si es posible, contra un despliegue de producción del backend.
- Diseñar y correr una prueba de carga con varios agentes reportando heartbeats y eventos al mismo tiempo, antes de proyectar el sistema sobre redes grandes.
- Con esos tres resultados, cerrar la sección 4 y 5 de este informe con datos medidos en vez de análisis de diseño.

Fuentes de datos: `rendimiento_backend.json` y `rendimiento_vite.json` (repositorio del proyecto, 22 de septiembre de 2026).
Herramientas: `tests/performance/benchmark_navigation.py` (existente) y `tests/performance/benchmark_agent_footprint.py` (nueva, sección 4).